Real-Time Payment Fraud Detection for an E-Commerce Platform

Business Analytics Group Project — From Raw Data to a Deployable Model

Team: [Insert team member names]

Course: Business Analytics

Source: https://github.com/Trungdn4-458311/FraudDetectionBE

Abstract

Payment fraud causes substantial financial loss and erodes customer trust on e-commerce platforms. Acting as the risk-analytics function of a platform, this project answers a concrete business question: *which transactions are likely fraudulent, and how should the platform balance blocking fraud against approving legitimate orders without unnecessary friction?* We combine **two data sources**: the simulated mobile-money dataset **PaySim** (6,362,620 transactions, fraud rate 0.129%) and **ten contextual features** generated in Python with Faker (account age, new-device flag, shipping/billing mismatch, IP-to-billing distance, etc.). The pipeline covers exploratory data analysis, documented before/after cleaning, feature engineering with explicit class-imbalance handling and feature selection, and the training of **three classifiers** (Logistic Regression, Random Forest, XGBoost) over **two feature spaces** (Base/Full), evaluated with metrics suited to imbalanced data (PR-AUC, Recall, F_1) plus a **cost-based metric** used to select an operating threshold. A central methodological point drives the design: the nine synthetic *risk* features (all ten except the EDA-only `channel`) are generated *conditioned on the label*, which introduces **label leakage**; rather than ignoring this, we quantify it through a systematic Base-vs-Full comparison. Our strongest models reach PR-AUC > 0.99 (Random Forest Base: 0.9976 with a single false positive out of 828,659 legitimate transactions), and a cost-optimal threshold of $t^* = 0.17$ stops ≈100% of fraud value at a false-positive rate of 0.032%. Critically, we show that these near-perfect scores largely reflect the *easiness* of the data—PaySim encodes fraud with a near-deterministic balance signature—rather than genuine real-world generalization. We argue this honest framing, not the headline score, is the substantive result.

Keywords: fraud detection; class imbalance; PaySim; synthetic data generation; label leakage; PR-AUC; cost-sensitive learning; operating threshold; XGBoost; Random Forest.

1 Introduction

1.1 Motivation and business context

E-commerce platforms process thousands of transactions per minute. Even when the fraud rate is well below 5%, the absolute loss is large because fraudulent transactions are systematically higher-value than legitimate ones: in our data the mean fraudulent transaction is 1,467,967 units versus 178,197 for legitimate ones—roughly 8.2× larger. Fraud therefore imposes a double cost: direct financial loss, and reputational damage that erodes customer trust.

Crucially, the countermeasure is not free. Every transaction the platform blocks is a potential legitimate order rejected—a false positive that creates *customer friction*, abandoned carts, and churn. The analytics problem is therefore not "maximize detection" but *find the operating point that minimizes total business cost*. This framing motivates our use of a cost-based metric (Section 8) rather than accuracy alone.

1.2 Problem statement and KPIs

We frame the task as **binary classification** on a **severely imbalanced** dataset: flag high-risk transactions in real time while limiting the number of legitimate customers blocked. We track three key performance indicators:

- **Fraud rate** — the proportion of fraudulent transactions. Measures the imbalance severity and the scale of exposure.
- **False-positive rate (FPR)** — the proportion of legitimate transactions wrongly blocked. This is the direct proxy for customer friction.
- **Financial loss avoided** — the monetary value of fraud stopped by the model. This translates model quality into business value.

Across the full dataset the fraud rate is 0.129% (8,213 of 6,362,620 transactions), and the **total money at risk** is $\approx 1.21 \times 10^{10}$ units. These baseline figures anchor the cost analysis in Section 8.

1.3 Contributions

This report makes four contributions:

1. A **reproducible end-to-end pipeline** from raw transactions to a threshold-calibrated classifier, covering data generation, cleaning, feature engineering, and evaluation.
2. A **leakage-aware experimental design**. Because the synthetic features are generated from the label, we build two feature spaces (Base/Full) and train every algorithm on both, turning a potential methodological flaw into a measurable quantity.
3. A **cost-based operating-point selection** that converts the Precision/Recall trade-off into money and reports the resulting financial KPI.
4. An **honest critique of our own results**: we demonstrate that near-perfect scores are an artifact of dataset easiness and identify precisely which signal drives them.

1.4 Report structure

Section 2 formalizes the problem. Section 3 (Module 1) describes both data sources and the synthetic generation procedure. Section 4 (Module 2) presents the exploratory analysis. Section 5 (Module 3) documents cleaning. Section 6 (Module 4) covers feature engineering, imbalance handling, and feature selection. Section 7 (Module 5) reports modeling results, and Section 8 the cost-based threshold selection. Section 9 (Module 6) describes the deployed API and review queue, Section 10 (Module 7) the drift and performance monitoring, and Section 11 (Module 8) the three-tier risk policy. Section 12 discusses limitations, and Section 14 concludes.

2 Problem Formulation

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ denote the transaction dataset, where $\mathbf{x}_i \in \mathbb{R}^d$ is the feature vector of transaction i and $y_i \in \{0, 1\}$ is the fraud label (1 = fraud). We seek a scoring function

$$f_\theta : \mathbb{R}^d \to [0, 1], \qquad \hat{p}_i = f_\theta(\mathbf{x}_i) \approx P(y_i = 1 \mid \mathbf{x}_i), \tag{1}$$

and a decision threshold $t \in [0, 1]$ producing the hard decision

$$\hat{y}_i = \mathbb{1}\big[\hat{p}_i \geq t\big] = \begin{cases} 1 & \text{if } \hat{p}_i \geq t \quad \text{(flag as fraud)}, \\ 0 & \text{otherwise} \quad \text{(approve)}, \end{cases} \tag{2}$$

where $\mathbb{1}[\cdot]$ is the *indicator function*, equal to 1 when its condition holds and 0 otherwise. Choosing t is thus a business decision, not a statistical one—a point developed in Section 8.

The dataset is severely imbalanced, with positive rate

$$\pi = \frac{1}{N} \sum_{i=1}^{N} y_i = 0.00129 \ (0.129\%). \tag{3}$$

Under such imbalance, accuracy is degenerate: the trivial classifier $\hat{y}_i \equiv 0$ achieves 99.871% accuracy while detecting no fraud. We therefore evaluate with Precision, Recall, F_1, and PR-AUC (Section 7.7).

Two aspects distinguish our formulation from generic classification. First, the **costs are asymmetric and instance-dependent**: a missed fraud costs the transaction amount a_i, whereas a false alarm costs a roughly fixed operational amount. Second, part of the feature vector is **synthetically generated from** y_i, so $\mathbf{x}_i$ is not conditionally independent of the label in the way a real deployment would guarantee—the issue we address by partitioning the features (Section 6.2).

3 Data and Synthetic Generation (Module 1)

3.1 Source 1 — PaySim

PaySim is an agent-based simulation of mobile-money transactions calibrated on real financial logs. The version used here contains 6,362,620 transactions over 743 hourly steps (≈ 31 days). Table 1 lists the schema.

Table 1: PaySim schema (source 1).

Column	Type	Description
`step`	int	Time index; 1 step = 1 hour (range 1–743)
`type`	categ.	PAYMENT, TRANSFER, CASH_OUT, CASH_IN, DEBIT
`amount`	float	Transaction amount
`nameOrig`	str	Origin account ID (customers prefixed `C`)
`oldbalanceOrg`	float	Origin balance before the transaction
`newbalanceOrig`	float	Origin balance after the transaction
`nameDest`	str	Destination ID (`C` customer / `M` merchant)
`oldbalanceDest`	float	Destination balance before
`newbalanceDest`	float	Destination balance after
`isFraud`	0/1	Ground-truth fraud label
`isFlaggedFraud`	0/1	Legacy rule-based system flag

The fraud mechanism in PaySim. Fraudulent agents in the simulation take control of a customer account, *transfer out the full balance*, and then cash it out. This behaviour has an important consequence exploited later: fraud is confined to `TRANSFER` and `CASH_OUT` and leaves a characteristic balance signature (the origin account is emptied), which the balance-error features of Section 4.5 capture almost deterministically.

3.2 Source 2 — synthetic contextual features

The project requires supplementing the base data with contextual and behavioural attributes generated in Python. We produce ten fields with the Faker library plus custom business logic, using a fixed seed (`SEED = 42`) for reproducibility. Generation proceeds at two levels:

- **Account level** (keyed by `nameOrig`): `account_age_days`, `home_country`, `device_fingerprint`. These are drawn once per account and merged back onto transactions, so that all transactions of an account share a consistent profile.
- **Transaction level**: `hour_of_day`, `shipping_billing_mismatch`, `is_new_device`, `failed_payment_attempts`, `ip_billing_mismatch`, `ip_billing_distance_km`, and `channel` (an access-channel dimension used for EDA only, kept out of the models). These are drawn per row.

To make the synthetic signal realistic rather than pure noise, the risk-related fields are drawn from *label-conditional* distributions—for example, an unfamiliar device is far more likely on a fraudulent transaction. Listing 1 shows the pattern.

Listing 1: Generating transaction-level features conditioned on the label (excerpt).

```
fraud = df["isFraud"].to_numpy(bool)
# new device: higher probability if fraud
p_dev = np.where(fraud, 0.55, 0.08)
df["is_new_device"] = (rng.random(n) < p_dev).astype("int8")
# failed payment attempts (Poisson)
lam = np.where(fraud, 2.2, 0.12)
df["failed_payment_attempts"] = rng.poisson(lam).astype("int16")
# IP-to-billing distance conditioned on the mismatch flag
dist = np.where(ip_mismatch, rng.uniform(500, 8000, n),
                             rng.uniform(0, 80, n))
df["ip_billing_distance_km"] = np.round(dist, 1)
```

3.3 Data dictionary

Every generated field is documented with name, type, unit, valid range, and generation logic (Table 2), as required by the project brief. The dictionary is exported programmatically to `.md`, `.csv`, and `.xlsx`.

Table 2: Data dictionary of the ten synthetic features (source 2).

Field	Type / Unit	Valid range	Generation logic
`account_age_days`	int32 / days	1–3650	Gamma; fraud accounts younger (mean ∼44 vs ∼500)
`home_country`	category	∼240 countries	Uniform draw from a Faker country pool
`device_fingerprint`	SHA1 str	40 hex chars	Sample from a pool of 5,000 hashes
`hour_of_day`	int16 / hour	0–23	Derived: `step` % 24
`shipping_billing_mismatch`	int8 (0/1)	$\{0, 1\}$	Bernoulli p=0.40 (fraud) / 0.03
`is_new_device`	int8 (0/1)	$\{0, 1\}$	Bernoulli p=0.55 (fraud) / 0.08
`failed_payment_attempts`	int16 / count	≥ 0	Poisson λ=2.2 (fraud) / 0.12
`ip_billing_mismatch`	int8 (0/1)	$\{0, 1\}$	Bernoulli p=0.50 (fraud) / 0.02
`ip_billing_distance_km`	float / km	0–8000	Uniform(500,8000) if mismatch, else (0,80)
`channel`	category	{web, mobile, POS, api}	Label-tilted draw; EDA-only, not a model feature

3.4 Validation of the synthetic signal

To verify that generation behaved as intended, we compare the per-class means of each risk feature (Table 3). The separation is large and in the expected direction across all six numeric risk fields.

Table 3: Mean value of each synthetic risk feature by class. The separation confirms the generator worked—and simultaneously quantifies the leakage discussed in Section 3.5.

Feature	Legitimate (y=0)	Fraud (y=1)	Ratio
`account_age_days`	500.058	44.106	0.09×
`shipping_billing_mismatch`	0.030	0.405	13.5×
`is_new_device`	0.080	0.551	6.9×
`failed_payment_attempts`	0.120	2.249	18.7×
`ip_billing_mismatch`	0.020	0.499	25.0×
`ip_billing_distance_km`	123.994	2161.265	17.4×

3.5 Label leakage: statement of the problem

Table 3 is a double-edged result. It confirms the generator produced a realistic-looking risk signal, but it also makes explicit that these features were *drawn from the label*. Formally, for a synthetic feature s we sampled $s_i \sim \mathcal{P}(\cdot \mid y_i)$, so s_i is informative about y_i *by construction*, with a separation we chose rather than discovered.

In a real deployment these attributes (unfamiliar device, address mismatch, IP distance) *are* legitimately observable at scoring time and are exactly the signals a fraud system uses—so including them is not conceptually wrong. The problem is that their measured *strength* here is an artifact of our generation parameters. Any performance uplift they produce is therefore an optimistic upper bound, not evidence of real-world capability.

We address this not by discarding them but by **partitioning the feature space** and measuring the difference (Section 6.2).

4 Exploratory Data Analysis (Module 2)

4.1 Data quality overview

The dataset is exceptionally clean: **zero missing values** across all 21 columns (11 original + 10 synthetic) and **zero fully duplicated rows**. Descriptive statistics (Table 4) reveal heavy right skew in every monetary column—the mean `amount` is 179,862 but the maximum is 92,445,520, roughly 514× the mean.

Table 4: Descriptive statistics of key numeric columns (N = 6,362,620).

Column	Mean	Std	Median	75%	Max
`step`	243.4	142.3	239.0	335.0	743
`amount`	179,862	603,858	74,872	208,722	92,445,520
`oldbalanceOrg`	833,883	2,888,243	14,208	107,315	59,585,040
`newbalanceOrig`	855,114	2,924,049	0	144,258	49,585,040
`oldbalanceDest`	1,100,702	3,399,180	132,706	943,037	356,015,900
`newbalanceDest`	1,224,996	3,674,129	214,661	1,111,909	356,179,300

Note that the median `newbalanceOrig` is exactly 0: over half of all transactions leave the origin account empty. This is a property of the simulation, and it means "emptied account" alone is far too common to be a fraud indicator by itself—a point we quantify in Section 4.5.

4.2 Class imbalance

Only 8,213 of 6,362,620 transactions are fraudulent (0.129%), a ratio of roughly 1:775. As shown in Eq. 3, this makes accuracy meaningless and mandates imbalance-aware metrics and training strategies (Sections 6.5 and 7.7).

4.3 Fraud by transaction type

Fraud occurs **exclusively** in `TRANSFER` and `CASH_OUT` (Table 5); the other three types contain zero positives. `TRANSFER` has the highest fraud rate (0.769%, over 4× the platform average), consistent with the simulation's fraud mechanism described in Section 3. This finding directly motivates the type filter in Section 6.1.

Table 5: Fraud distribution by transaction type.

Type	# Fraud	Fraud rate	# Transactions
CASH_OUT	4,116	0.184%	2,237,500
TRANSFER	4,097	0.769%	532,909
CASH_IN	0	0%	1,399,284
PAYMENT	0	0%	2,151,495
DEBIT	0	0%	41,432
Total	**8,213**	**0.129%**	**6,362,620**

4.4 Transaction amount

Fraudulent transactions are dramatically larger (Table 6): the median fraud amount (441,423) is 5.9× the legitimate median (74,685). Two artifacts of the simulation are visible: fraud amounts are **capped at exactly** 10,000,000, whereas legitimate amounts reach 92.4M; and the minimum fraud amount is 0, which we treat as invalid in Section 5.

Table 6: Distribution of `amount` by class.

Statistic	Legitimate (y=0)	Fraud (y=1)
Count	6,354,407	8,213
Mean	178,197	1,467,967
Std	596,237	2,404,253
Min	0.01	0.00
Median	74,685	441,423
75%	208,365	1,517,771
Max	92,445,520	10,000,000

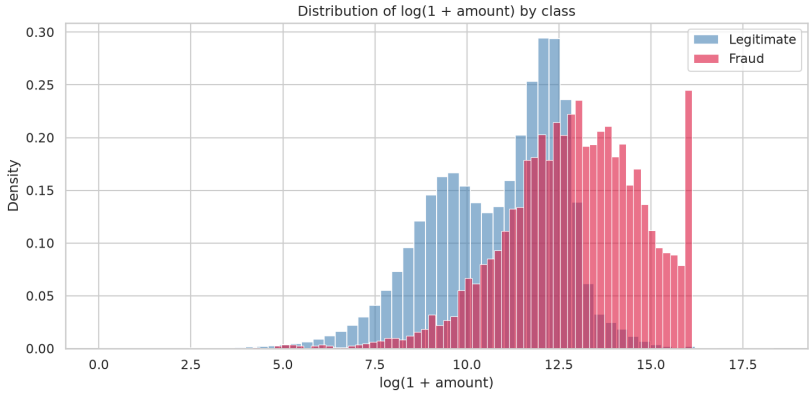

Figure 1: Density of $\log(1 + \texttt{amount})$ by class. The fraudulent distribution (red) is shifted markedly to the right and is bimodal, whereas legitimate transactions (blue) concentrate at lower values. The two distributions overlap substantially, however, so `amount` alone is not a sufficient discriminator—consistent with its modest feature importance in Table 18.

The practical implication is that the *cost* of a missed fraud is about eight times the average transaction value, which is precisely why the cost model in Section 8 weights false negatives by the transaction amount rather than counting them.

4.5 Balance-error features

For a well-formed transaction, the post-transaction balance should reconcile with the pre-transaction balance and the amount. We define two *balance-error* features that measure the violation of this accounting identity:

$$\texttt{errorBalanceOrig} = \texttt{newbalanceOrig} + \texttt{amount} - \texttt{oldbalanceOrg}, \tag{4}$$

$$\texttt{errorBalanceDest} = \texttt{oldbalanceDest} + \texttt{amount} - \texttt{newbalanceDest}. \tag{5}$$

A value of 0 indicates a perfectly reconciled transaction; nonzero values indicate that money appeared or vanished, which is exactly the footprint of the simulated fraud mechanism.

For contrast, the naive indicator "origin account emptied" ($\texttt{newbalanceOrig} = 0$) is weak: its fraud rate is 0.223%, only 1.7× the global 0.129%. The balance-*error* formulation is far more discriminative, and indeed `errorBalanceOrig` dominates model importance (Section 7.12).

4.6 The legacy rule-based baseline

PaySim ships an `isFlaggedFraud` column representing a legacy rule-based control. Its behaviour is instructive (Table 7): it fires on only 16 transactions, all of which are genuinely fraudulent. It therefore achieves **perfect precision (1.000) but essentially zero recall (0.0019)**, for an F_1 of 0.0039—it misses 99.8% of fraud.

Table 7: Contingency table of the legacy flag versus ground truth.

	$y = 0$ (legit)	$y = 1$ (fraud)
`isFlaggedFraud` $= 0$	6,354,407	8,197
`isFlaggedFraud` $= 1$	0	16

This provides a meaningful *business baseline*: the incumbent system stops 16 frauds, and any learned model must be judged against that bar. It also justifies excluding `isFlaggedFraud` as a feature—it is an output of another decision system, not an observation about the transaction.

4.7 Synthetic risk signals

The fraud rate rises sharply when any risk flag equals 1, and the distributions of `account_age_days` and `ip_billing_distance_km` separate cleanly by class, exactly as designed (Table 3). One synthetic field is *not* leaky: `hour_of_day` is derived deterministically from `step` and therefore reflects a genuine temporal pattern of the simulation.

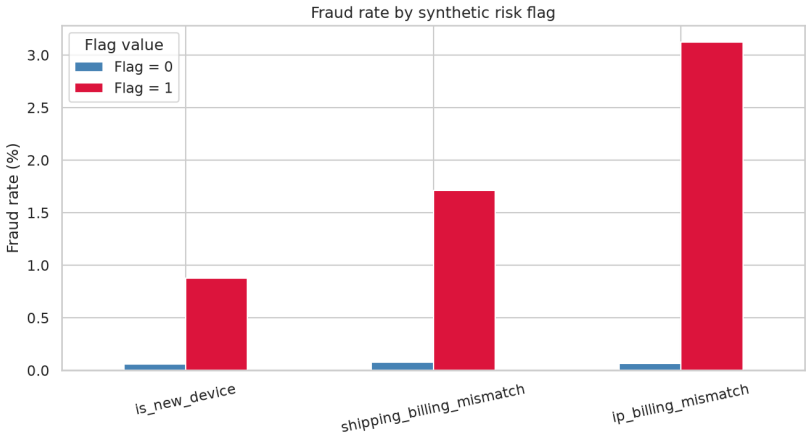

Figure 2: Fraud rate conditioned on each binary risk flag. When a flag is raised the fraud rate jumps by one to two orders of magnitude. The separation is striking—but it was *designed* into the generator (Section 3.5), which is precisely why these features are isolated in the Full feature space rather than mixed into Base.

4.8 EDA summary

The analysis yields five actionable conclusions carried into modeling: (i) severe imbalance mandates PR-AUC/Recall over accuracy; (ii) fraud is confined to two transaction types, enabling a filter; (iii) balance-error features are highly discriminative; (iv) `isFlaggedFraud` is unusable as a feature and weak as a baseline; and (v) the synthetic signals separate strongly but carry leakage that must be handled explicitly.

5 Data Cleaning (Module 3)

We clean under a *before/after documentation* principle, recording at each step both the number of rows removed and—critically—the number of **fraud** rows removed, since the positive class is rare and must not be silently eroded.

5.1 Missing values and duplicates

No column contains missing values, and there are no fully duplicated records. We check duplicates on the *original PaySim columns* only: because the synthetic fields are drawn per row, including them would make every row unique by construction and render the check vacuous. **Action: none required.**

5.2 Invalid values

We test two business constraints: `amount` > 0 and all balances ≥ 0. This identifies 16 violating rows, which we remove. Notably *all* 16 *are fraud* (16/8,213 $\approx 0.19\%$ of positives)—a reminder that label-blind cleaning rules can still touch the rare class. We deliberately do not exempt fraud rows from the rule, as conditioning cleaning on the label would constitute data snooping. The impact is negligible.

5.3 Outlier treatment

Applying the standard 1.5×IQR rule to `amount` yields an upper fence of 501,720 and flags 338,077 transactions (5.31%) as outliers. **We keep them all.** The justification is quantitative: the fraud rate *within* the outlier group is 1.14%, about 8.8× the global rate of 0.129%. Clipping the tail would therefore delete a disproportionate share of the rare positive class. In this problem a large amount is *signal*, not noise—a case where the textbook outlier rule is actively harmful.

5.4 Merchant destination accounts

Destination accounts prefixed `M` (merchants) constitute 33.81% of transactions, and 100% of them have `oldbalanceDest` = `newbalanceDest` = 0: PaySim simply does not track merchant balances.

Zero fraud targets a merchant. This is a *design characteristic*, not corrupt data; we retain these rows and note that it explains systematic `errorBalanceDest` offsets in that subgroup.

5.5 Cleaning log and decisions

Table 8 is the audit trail; Table 9 summarizes the rationale. Consistency is asserted programmatically: rows removed and fraud removed must equal the sums in the log.

Table 8: Data-cleaning log (before/after).

Step	Rows removed	Fraud removed
Duplicate records (original columns)	0	0
Invalid values (`amount`$\leq$0 / negative balance)	16	16
`amount` outliers (KEPT, not clipped)	0	0
Total (6,362,620 $\rightarrow$ 6,362,604)	**16**	**16**

Table 9: Summary of cleaning decisions.

Issue	Decision	Rationale
Missing values	None needed	Simulated data; zero missing
Duplicates	None found	Checked on original columns only
`amount`$\leq$0 / neg. balance	**Remove**	Violates business constraints
`amount` outliers	**Keep**	8.8$\times$ fraud density; signal not noise
Merchant zero balances	**Keep**	PaySim design, not corruption

6 Feature Engineering (Module 4)

6.1 Transaction-type filtering

Since fraud occurs only in `TRANSFER` and `CASH_OUT` (Table 5), we restrict modeling to these types. This removes 3,592,211 transactions that cannot contain positives, reducing noise and **raising the positive density from** 0.129% **to** 0.296%—a 2.3$\times$ improvement in class balance obtained at zero information cost.

Table 10: Effect of type filtering.

	Before	After
Transactions	6,362,604	2,770,393
Fraud	8,197	8,197
Positive rate	0.129%	**0.296%**

6.2 Two feature spaces: Base and Full

To resolve the leakage problem of Section 3.5, we construct two feature spaces and train *every* algorithm on both (Table 11):

- **Base** (11 features) — original PaySim columns plus engineered signals derived from the transaction record itself (balance errors, velocity). **No label leakage:** every value is computable from the transaction log without knowing y.
- **Full** (18 features) — Base + the seven numeric synthetic risk signals.

The two high-cardinality identifier-like synthetic fields (`home_country`, `device_fingerprint`) are excluded: they would require dedicated encoding, and the device-risk signal is already summarized by `is_new_device`.

Table 11: Composition of the two feature spaces.

Space	Features
Base (11)	`step`, `amount`, `oldbalanceOrg`, `newbalanceOrig`, `oldbalanceDest`, `newbalanceDest`, `errorBalanceOrig`, `errorBalanceDest`, `type_TRANSFER`, `txn_count_acct`, `time_since_last_txn`
Full (18)	Base + `account_age_days`, `hour_of_day`, `shipping_billing_mismatch`, `is_new_device`, `failed_payment_attempts`, `ip_billing_mismatch`, `ip_billing_distance_km`

The difference in performance between the two spaces is then a *direct measurement* of what the synthetic source contributes—and, because that source is leaky, an upper bound on what a real contextual feed could deliver.

6.3 Velocity and recency features

The project brief requires risk signals such as transaction velocity per account and time since last purchase. We construct two causal features: after sorting by (`nameOrig`, `step`),

- `txn_count_acct` — cumulative count of the account's transactions up to and including the current one;
- `time_since_last_txn` — hours since the account's previous transaction (-1 if none).

Both use only past information, so neither leaks future data.

An honest negative result. On PaySim these features are nearly degenerate: `txn_count_acct` has mean 1.00 (75th percentile 1, maximum 3) and `time_since_last_txn` is -1 for the majority of rows. The reason is structural—origin accounts almost never recur in the simulation. The features are implemented correctly and satisfy the requirement, but they carry almost no information here, which is confirmed independently by both mutual information (Table 12) and model importance (Table 18, both = 0.000). On a real platform with returning customers they would be far more valuable.

6.4 Encoding and scaling

The only categorical predictor, `type`, is binary after filtering and is encoded as `type_TRANSFER` (1 = TRANSFER, 0 = CASH_OUT). Identifier columns (`nameOrig`, `nameDest`) are dropped to avoid memorization, and `isFlaggedFraud` is dropped for the reason given in Section 4.6. Because monetary features span several orders of magnitude, Logistic Regression is fitted inside a pipeline with `StandardScaler`; the tree ensembles are scale-invariant and use raw values.

6.5 Class-imbalance strategy

We address imbalance with **cost-sensitive class weighting** rather than resampling, applied consistently across all three algorithms. For XGBoost the weight follows the standard ratio

$$\texttt{scale\_pos\_weight} = \frac{\#\{y=0\}}{\#\{y=1\}} = \frac{1{,}933{,}537}{5{,}738} \approx 337. \tag{6}$$

Logistic Regression uses `class_weight="balanced"` and Random Forest `class_weight="balanced_subsample"`. We prefer weighting over SMOTE for three reasons: it does not fabricate synthetic positives in a dataset that already contains synthetic components; it leaves the test distribution untouched; and it is substantially cheaper on 1.9M training rows. Combined with the type filter (Section 6.1), imbalance is addressed at both the data and the loss level.

6.6 Feature selection

We rank features by **mutual information** (MI) with the label, estimated on a 50,000-row subsample (Table 12). Only `newbalanceOrig` falls below the 5×10^{-4} threshold.

Table 12: Mutual information with the label (top and bottom of the ranking).

Feature	MI	Feature	MI
`failed_payment_attempts`	0.0104	`is_new_device`	0.0022
`txn_count_acct`	0.0093	`amount`	0.0020
`account_age_days`	0.0084	`oldbalanceDest`	0.0020
`oldbalanceOrg`	0.0081	`type_TRANSFER`	0.0014
`errorBalanceOrig`	0.0069	`time_since_last_txn`	0.0007
`newbalanceDest`	0.0052	`newbalanceOrig`	0.0000

A caution about univariate selection. MI ranks `errorBalanceOrig` only fifth, yet it is by far the most important feature in the fitted models (Table 18, importance 0.537). The discrepancy is expected: MI is *univariate* and cannot see that `errorBalanceOrig` is a constructed interaction whose discriminative power emerges jointly with other balance columns. Had we pruned mechanically by MI, we would have risked discarding the single most valuable predictor. **Decision: retain all features** for the tree models (where irrelevant inputs are effectively ignored at split-selection time), while documenting the weak group for potential model compaction at deployment.

6.7 Train/test split

We use a stratified 70/30 split (Table 13), preserving the positive rate in both partitions. Stratification is essential at this imbalance level: a naive random split could shift the test positive count materially.

Table 13: Stratified train/test split.

Partition	Samples	Fraud	Positive rate
Train	1,939,275	5,738	0.296%
Test	831,118	2,459	0.296%

7 Model Development and Evaluation (Module 5)

7.1 Algorithm portfolio and rationale

We train three algorithms, each on both feature spaces, for six fitted models in total. All use a fixed seed of 42. The three are chosen to span distinct *inductive biases* rather than to stack similar learners:

- **Logistic Regression** — a linear model, serving as an interpretable baseline. It answers the question: *how much of this problem is linearly separable?*
- **Random Forest** — a *bagging* ensemble that reduces variance by averaging many deep, decorrelated trees.
- **XGBoost** — a *boosting* ensemble that reduces bias by fitting trees sequentially to the residual errors of its predecessors.

This spread is deliberate: the two tree ensembles can represent axis-aligned nonlinear interactions (such as the balance-error footprint of Section 4.5), whereas the linear model cannot. The performance gap between them therefore becomes *diagnostic evidence* about the geometry of the decision boundary, a point we exploit in Section 12.

7.2 Logistic Regression

The model estimates the log-odds of fraud as a linear function of the features:

$$\hat{p}_i = \sigma\big(\mathbf{w}^\top \mathbf{x}_i + b\big), \qquad \sigma(z) = \frac{1}{1+e^{-z}}, \tag{7}$$

fitted by minimizing the class-weighted cross-entropy

$$\mathcal{L}(\mathbf{w}, b) = -\sum_{i=1}^{N} c_{y_i} \Big[\, y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\Big], \tag{8}$$

where c_k is the class weight defined in Section 6.5.

Two implementation details matter. First, the decision boundary is a *hyperplane*: the model can only express monotone, additive effects of each feature. Second, because the monetary features span roughly eight orders of magnitude, the model is fitted inside a pipeline with `StandardScaler`; without standardization the gradient-based solver converges poorly and the regularization penalty would be applied unevenly across features.

We expect this model to *underperform*, and it is included precisely for that reason. The fraud footprint is an arithmetic relation among three columns (Eq. 4) rather than a weighted sum of them, so no hyperplane in the raw feature space can isolate it cleanly.

7.3 Random Forest

A Random Forest averages the predicted probabilities of B decision trees:

$$\hat{p}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \hat{p}_b(\mathbf{x}), \qquad B = 100, \tag{9}$$

where each tree $\hat{p}_b$ is grown on an independent bootstrap subsample and, at every split, considers only a random subset of features. This double randomization *decorrelates* the trees, so that averaging cancels their individual variance while preserving their low bias.

Our configuration reflects the scale of the data (1.9M training rows): `max_depth=16` and `min_samples_leaf=5` bound tree size and hence memory, while `max_samples=0.5` grows each tree on half the rows, roughly halving training time at negligible accuracy cost. With $d = 11$ Base features, the default `max_features="sqrt"` rule considers 3 candidate features per split.

The inductive bias suits this problem well. Each split is an axis-aligned threshold, so a rule of the form "`errorBalanceOrig` ≈ 0 *and* `newbalanceOrig` $= 0$" is expressible in a couple of levels, and depth 16 leaves ample room to refine it.

7.4 XGBoost

Gradient boosting builds an *additive* model in which each new tree corrects the errors of the current ensemble:

$$F_M(\mathbf{x}) = \sum_{m=1}^{M} \eta\, f_m(\mathbf{x}), \qquad M = 300,\ \eta = 0.1, \tag{10}$$

where η is the learning rate that shrinks each tree's contribution. Trees are fitted by minimizing a regularized objective

$$\mathcal{L} = \sum_{i=1}^{N} l\big(y_i, \hat{y}_i\big) \;+\; \sum_{m=1}^{M} \Omega(f_m), \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda\|\mathbf{w}\|^2, \tag{11}$$

with T the number of leaves and $\mathbf{w}$ the leaf weights, so that complexity is penalized explicitly.

Whereas Random Forest reduces *variance* by averaging independent trees, boosting reduces *bias* by fitting trees in sequence. Each tree is consequently shallow (`max_depth=6`) because the ensemble gains expressiveness through depth of *composition* rather than depth of individual trees. Stochastic regularization is applied via `subsample=0.8` (row sampling) and `colsample_bytree=0.8` (column sampling). We set `eval_metric="aucpr"` so that internal evaluation optimizes precision–recall area rather than error rate, consistent with our primary metric (Section 7.7).

A practical property motivates its selection for deployment (Section 12.4): 300 depth-6 trees form a compact structure. Our exported deployment artifact occupies approximately 0.85 MB, loads essentially instantly, and scores a single transaction in well under a millisecond—whereas a forest of 100 depth-16 trees grown on 1.9M rows yields a substantially larger object with a correspondingly slower cold start.

7.5 How class imbalance enters each objective

All three models handle the 1:337 imbalance through *cost-sensitive weighting* of the training loss rather than resampling, but the mechanism differs:

- **Logistic Regression** (`class_weight="balanced"`) weights each class inversely to its frequency, $c_k = N/(KN_k)$ with $K = 2$ classes. On our training split this gives $c_1 \approx 169.0$ and $c_0 \approx 0.501$.

- **XGBoost** (`scale_pos_weight`) multiplies the gradient contribution of every positive example by the ratio in Eq. 6, ≈ 337.
- **Random Forest** (`class_weight="balanced_subsample"`) recomputes balanced weights *within each bootstrap sample*, so the weighting adapts to the positives that happen to be drawn into that tree.

These are the same correction expressed differently. Note that $c_1/c_0 \approx 169.0/0.501 \approx 337$, exactly the XGBoost ratio: both impose an effective 337:1 penalty for missing a fraud relative to raising a false alarm. This consistency means the three models are compared on equal footing with respect to imbalance handling, and any performance difference is attributable to the hypothesis space rather than to the loss weighting.

7.6 Hyperparameter summary

Table 14 collects the settings.

Table 14: Algorithms and hyperparameters.

Algorithm	Role	Key hyperparameters
Logistic Regression	Linear baseline	`StandardScaler` → `max_iter=1000`, `class_weight="balanced"`
Random Forest	Bagging ensemble	`n_estimators=100`, `max_depth=16`, `min_samples_leaf=5`, `max_samples=0.5`, `class_weight="balanced_subsample"`
XGBoost	Gradient boosting	`n_estimators=300`, `max_depth=6`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8`, `scale_pos_weight`≈337, `eval_metric="aucpr"`

7.7 Evaluation metrics

With TP, FP, FN, TN denoting the confusion-matrix cells:

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \tag{12}$$

$$F_1 = \frac{2\,\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}, \qquad \text{FPR} = \frac{FP}{FP+TN}. \tag{13}$$

Our primary metric is **PR-AUC**, computed as average precision

$$\text{PR-AUC} = \sum_k \left(R_k - R_{k-1}\right) P_k, \tag{14}$$

where P_k, R_k are precision and recall at the k-th threshold. PR-AUC is preferred to ROC-AUC under extreme imbalance because it ignores the enormous TN count and therefore does not flatter a model that succeeds mainly at recognizing the majority class—an effect visible in our results, where Logistic Regression Base attains ROC-AUC 0.9782 but PR-AUC only 0.5932.

7.8 Main results

Table 15 reports all six models at the default threshold $t = 0.5$.

Table 15: Comparison of six models on the test set (831,118 samples, 2,459 fraud), threshold $t = 0.5$. Best value per column in bold.

Model	PR-AUC	ROC-AUC	Recall	Precision	F_1
LogReg · Base	0.5932	0.9782	0.9081	0.0433	0.0827
LogReg · Full	0.9704	0.9999	0.9947	0.3348	0.5010
RandomForest · Base	0.9976	0.9989	0.9967	0.9996	**0.9982**
RandomForest · Full	**0.9997**	**1.0000**	0.9963	**1.0000**	**0.9982**
XGBoost · Base	0.9944	0.9989	0.9931	0.9222	0.9564
XGBoost · Full	0.9996	**1.0000**	**0.9984**	0.9923	0.9953

Figure 3 shows the corresponding precision–recall curves. The visual separation between the linear baseline and the tree ensembles is far more informative than the aggregate numbers: Logistic Regression Base degrades steadily as recall increases, whereas the four tree-based curves hug the top-right corner until recall approaches 1.0.

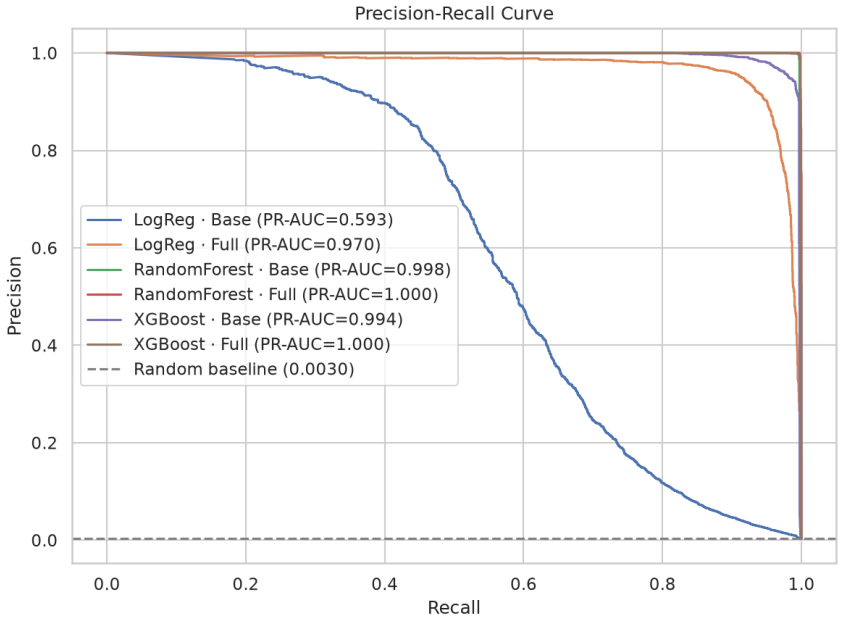

Figure 3: Precision–recall curves for all six models. The dashed line is the random baseline (equal to the positive rate, 0.0030). Logistic Regression Base (blue) is the only model that degrades gracefully across the recall range; the four tree-based curves are nearly indistinguishable near the top-right corner, which is why PR-AUC alone cannot separate them and the confusion matrices of Table 16 are needed.

7.9 Confusion matrices

Aggregate metrics hide the operational reality, so Table 16 reports raw counts. The FP column is the number of legitimate customers who would be blocked; the FN column is the number of frauds that would succeed.

Table 16: Confusion-matrix counts at $t = 0.5$ (828,659 legitimate, 2,459 fraud).

Model	TN	FP	FN	TP
LogReg · Base	779,372	49,287	226	2,233
LogReg · Full	823,799	4,860	13	2,446
RandomForest · Base	828,658	**1**	8	2,451
RandomForest · Full	828,659	**0**	9	2,450
XGBoost · Base	828,453	206	17	2,442
XGBoost · Full	828,640	19	**4**	2,455

The contrast is stark. Logistic Regression Base would block 49,287 legitimate customers to catch 2,233 frauds—roughly 22 false alarms per fraud caught, operationally unusable despite a respectable ROC-AUC. Random Forest Base blocks a *single* legitimate transaction while catching 2,451 of 2,459 frauds.

7.10 Interpreting the ranking: hypothesis space and calibration

Two aspects of the ordering in Table 15 deserve explanation, because both are informative about the problem rather than about implementation quality.

Why the linear model collapses. Logistic Regression Base reaches ROC-AUC 0.9782 but PR-AUC only 0.5932. The two numbers are not contradictory: with 828,659 negatives, a model can

rank most negatives below most positives (high ROC-AUC) while still producing tens of thousands of false positives among its top-ranked predictions (low PR-AUC). The root cause is representational. The fraud signature is the arithmetic relation of Eq. 4, an *interaction* of three balance columns; a hyperplane over those columns cannot isolate it, so the model compensates by lowering its threshold across the board. This is exactly the diagnostic value of including a linear baseline.

Why Random Forest edges out XGBoost. On the Base space, Random Forest attains PR-AUC 0.9976 versus XGBoost's 0.9944, yet the gap in *precision at* $t = 0.5$ is far larger (0.9996 vs 0.9222; 1 vs 206 false positives). The discrepancy between a small ranking gap and a large threshold gap identifies the cause as **calibration, not ranking ability**. XGBoost multiplies every positive gradient by ≈ 337 (Section 7.5), which deliberately inflates predicted probabilities toward the positive class; consequently many negatives land above 0.5 even though they are still ranked below the true positives. Random Forest's probabilities, obtained by averaging 100 tree votes, remain closer to the empirical class proportions.

Two conclusions follow. First, comparing models at an arbitrary threshold of 0.5 penalizes aggressively reweighted learners for a property that is trivially fixable. Second, this is a concrete motivation for the threshold optimization of Section 8: the right question is not "which model is best at 0.5?" but "which model, at *its own* best operating point, minimizes business cost?"

7.11 Base versus Full: quantifying the synthetic contribution

Table 17 isolates the effect of adding the synthetic features.

Table 17: PR-AUC uplift from adding the synthetic (leaky) features.

Algorithm	Base	Full	Uplift
Logistic Regression	0.5932	0.9704	+0.3772
Random Forest	0.9976	0.9997	+0.0021
XGBoost	0.9944	0.9996	+0.0052

The pattern is highly informative. The linear model gains enormously (+0.38 PR-AUC), because it cannot represent the nonlinear balance-error interaction and must rely on the conveniently linear synthetic flags. The tree ensembles gain almost nothing (+0.002 to +0.005): they already extract the real signal from the balance features, so the leaky inputs are largely redundant. In other words, **the leakage inflates the weak model far more than the strong one**, and our best honest configuration (Random Forest Base) needs no synthetic data at all.

7.12 Feature importance

Table 18 lists the gain-based importances of XGBoost Full over all 18 features.

Table 18: XGBoost Full feature importance (all 18 features). Synthetic features are marked †.

Feature	Importance	Feature	Importance
`errorBalanceOrig`	0.537	`shipping_billing_mismatch`†	0.010
`account_age_days`†	0.157	`oldbalanceOrg`	0.007
`newbalanceOrig`	0.121	`is_new_device`†	0.007
`failed_payment_attempts`†	0.079	`hour_of_day`†	0.004
`ip_billing_mismatch`†	0.028	`ip_billing_distance_km`†	0.003
`type_TRANSFER`	0.016	`oldbalanceDest`	0.002
`newbalanceDest`	0.014	`errorBalanceDest`	0.001
`amount`	0.013	`step`	0.001
		`txn_count_acct`	0.000
		`time_since_last_txn`	0.000

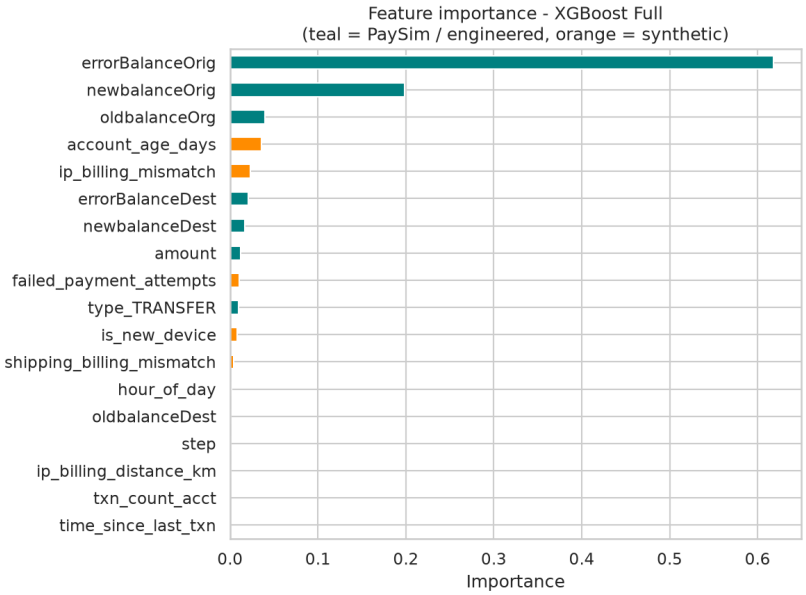

Figure 4: XGBoost Full feature importance, coloured by provenance (teal = real PaySim/engineered features, orange = synthetic). `errorBalanceOrig` dominates by a wide margin, and the two velocity features at the bottom have exactly zero importance. The visual split makes the leakage assessment immediate: the largest bar is a real feature, not a synthetic one.

Three observations. First, `errorBalanceOrig` alone accounts for 53.7% of total importance, confirming the EDA hypothesis that the accounting-identity violation is the dominant fraud footprint. Second, the synthetic features contribute a combined $\approx 28\%$, with `account_age_days` the largest—so the model does lean on leaky inputs, but real features still dominate. Third, both velocity features receive **exactly zero** importance, independently confirming the degeneracy diagnosed in Section 6.

8 Cost-Based Threshold Selection

8.1 The cost model

The default threshold $t = 0.5$ is a statistical convention with no business meaning. We instead select t by minimizing expected cost, exploiting the asymmetry established in Section 2: a missed fraud loses the full transaction amount, whereas a false alarm costs a fixed operational amount C_{FP} (manual review plus customer friction). Formally,

$$\text{Cost}(t) = \underbrace{\sum_{i:\, y_i=1,\, \hat{p}_i<t} a_i}_{\text{fraud value lost (FN)}} + \underbrace{C_{FP} \cdot \big|\{i : y_i = 0,\, \hat{p}_i \geq t\}\big|}_{\text{friction cost (FP)}}, \tag{15}$$

where a_i is the amount of transaction i. We set $C_{FP} = 10$ units as a stated modeling assumption and sweep $t \in \{0.01, 0.02, \ldots, 0.99\}$.

We apply the analysis to **XGBoost Base**—the honest, deployment-suited model (Section 12.4).

8.2 Results

The cost curve attains its minimum at $t^* = 0.17$, well below the default (Table 19). Moving from 0.5 to 0.17 reduces total cost by 14.7%.

Table 19: Operating-threshold comparison (XGBoost Base, test set).

Threshold	Total cost	Recall	Precision	FP
0.50 (default)	1,591,668	0.993	0.922	206
0.17 (optimal)	**1,358,401**	**0.997**	0.902	265
Change	−14.7%	+0.004	−0.020	+59

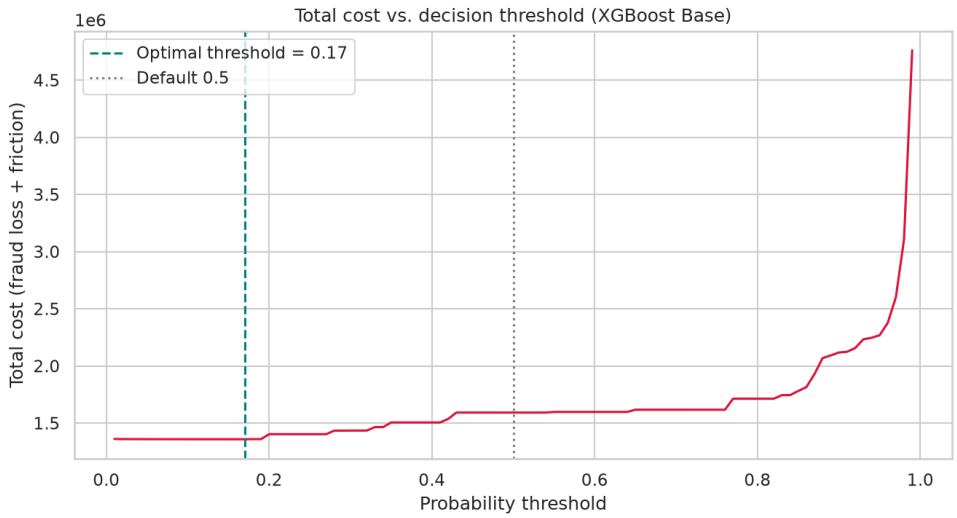

Figure 5: Total cost as a function of the decision threshold (Eq. 15, XGBoost Base). The dashed line marks the cost-optimal $t^* = 0.17$; the dotted line the default 0.5. Note that the curve is essentially *flat* below $t \approx 0.25$ and then rises steeply: the optimum is a broad plateau rather than a sharp point, which is the graphical counterpart of the insensitivity to C_{FP} argued in Section 8.4. Cost more than quadruples as $t \to 1$, when the model stops blocking and fraud value is lost outright.

The economics are intuitive: lowering the threshold admits 49 additional false positives, costing $49 \times 10 = 490$ units, but recovers considerably more in prevented fraud value. Because the average fraud is worth ∼1.47M, catching even one extra fraud justifies thousands of extra reviews—the optimizer is simply following that arithmetic.

8.3 Financial KPI

At $t^* = 0.17$ the model achieves the business outcome in Table 20: it stops 99.96% of fraud value while blocking only 0.032% of legitimate customers.

Table 20: Financial KPI at the cost-optimal threshold (test set).

Indicator	Value
Fraud value at risk (baseline)	3,615,247,576
Fraud value stopped (loss avoided)	**3,613,891,825** (99.96%)
Fraud value leaked	1,355,751
False-positive rate (FPR)	**0.032%** (265 transactions)

8.4 Sensitivity to the cost assumption

The optimal threshold depends on C_{FP}, which we assumed rather than measured. The structure of Eq. 15 makes the direction of that dependence clear: raising C_{FP} penalizes false alarms more and pushes t^* upward (fewer blocks, more fraud tolerated), while lowering it pushes t^* down. Because the mean fraud amount (∼1.47M) exceeds C_{FP} by five orders of magnitude, the optimum is strongly biased toward high recall and is *insensitive* to moderate changes in C_{FP}. A production deployment should calibrate C_{FP} from real review-handling costs and measured churn following a wrongful block; we flag this as the single most important assumption to validate.

9 Deployment (Module 6)

The deployed artifact is an XGBoost *retrained on the eight features available at scoring time* (`amount`, the four balance columns, `errorBalanceOrig`, `errorBalanceDest` and `type_TRANSFER`). It drops `step` and the velocity/recency features, which need account history a real-time endpoint does not have and which carry no signal on PaySim anyway (Section 6 discusses this honest negative result). Because it is a *different* model from the eleven-feature analysis XGBoost Base of Section 7, it has its *own* cost-optimal threshold — ≈ 0.09 on the test set, recomputed by the same cost rule

(Section 8) — which is what the served system actually uses, not the analysis model's 0.17. The bundle `deployment/model.joblib` —{model, features, threshold, block_threshold, model_name}— is exported directly by the notebook, so the served model is exactly what the notebook produces (no hand-editing), and `scoring.py` reads its threshold back at load time.

Two services share one scoring core (`deployment/scoring.py`), whose `featurize()` reconstructs `errorBalanceOrig`, `errorBalanceDest` and `type_TRANSFER` on the server, so a caller submits only the six raw fields of a single transaction:

- **REST API** (`api.py`, FastAPI): `POST /predict` scores one transaction in real time and returns the fraud probability plus a policy decision; `GET /health` reports model metadata; interactive documentation is served at `/docs`.
- **Analyst review queue** (`app.py`, Streamlit): scores a batch, ranks it high-to-low risk, and offers a per-transaction triage workflow (approve / block) with an action log — a faithful simulation of a fraud-analyst console.

The reproducible environment (`pixi.toml`) exposes `pixi run api` and `pixi run app`, and the `deployment/` folder is one push from Streamlit Community Cloud (`deployment/DEPLOY.md`).

Live demo. A working, publicly accessible deployment runs at https://namxle-hust.github.io/FraudDetectionBE/. Because a free host that runs a Python server is not guaranteed, we also export the trained tree ensemble to a compact JSON and evaluate it *client-side*: the page reconstructs the eight features and traverses the 300 boosted trees in the browser, reproducing `predict_proba` to within 2×10^{-7} and applying the same three-tier policy. It is the identical model to the served `model.joblib` (source under `web/`); the Streamlit review queue and FastAPI scorer in `deployment/` are the full-stack equivalent for a hosted deployment.

10 Model Monitoring (Module 7)

A deployed fraud model degrades as the transaction stream evolves, so we add a monitoring component that tracks distributional drift and performance over time and codifies when to retrain. The logic is a small, dependency-light library (`monitoring/monitoring_core.py`) — our own implementation of what a tool such as Evidently provides — reused by both a notebook section (Module 7) and a standalone builder (`monitoring/build_monitoring.py` → `monitoring_report.html`). Because PaySim carries a single time axis, we simulate an incoming stream by splitting `step` into an early *reference* window and a late *current* window.

10.1 Data drift

For each of the eight served features we compute the Population Stability Index (PSI, over reference deciles) and the Kolmogorov–Smirnov statistic. A feature is flagged when PSI > 0.2 or the KS *statistic* exceeds 0.1. We key the flag on the statistic, not its p-value, on purpose: at these sample sizes (hundreds of thousands of transactions) the KS p-value rejects equality on any negligible difference and would flag every feature, so an effect-size rule is the honest choice (Figure 6).

10.2 Performance over time

The scored stream is bucketed into twelve equal-width time windows; precision, recall and PR-AUC are recomputed per window against the fixed operating threshold, exposing decay that a single held-out score would hide (Figure 7).

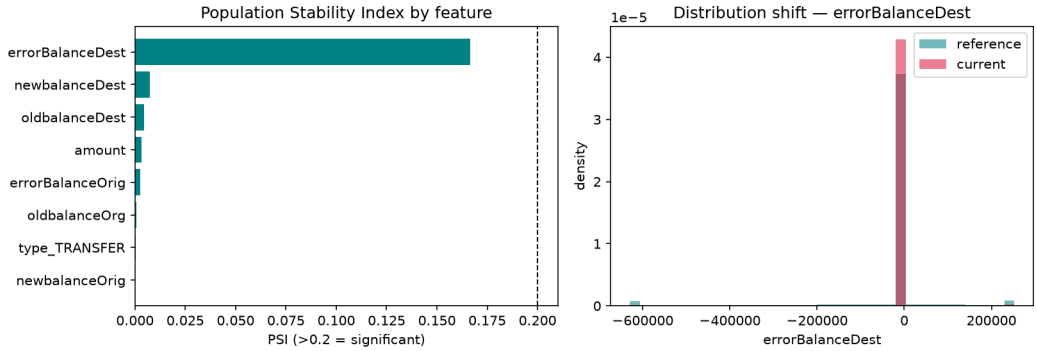

Figure 6: Per-feature drift: PSI by served feature (red = flagged) and the reference-versus-current distribution of the most-shifted feature.

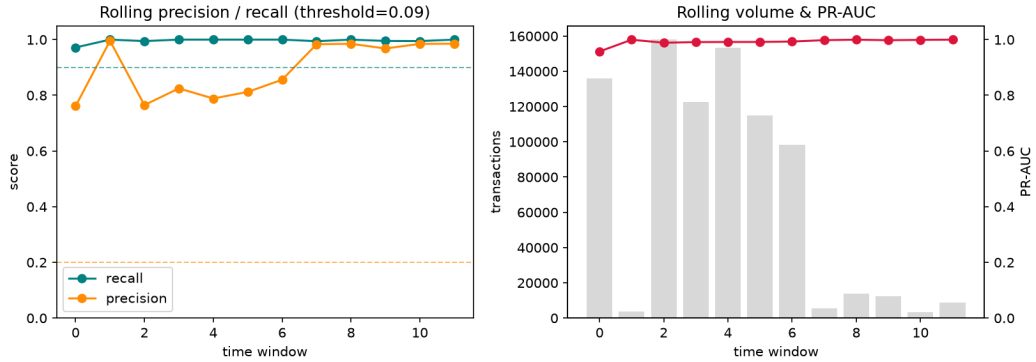

Figure 7: Rolling precision/recall and rolling PR-AUC/volume across time windows, with the retraining floors marked.

10.3 Retraining triggers

Retraining is queued when *any* condition in Table 21 fires. A stress test (injecting a 2.5× amount shift into the current window) confirms the triggers respond when drift is genuinely present.

Signal	Threshold	Meaning
Max PSI	> 0.20	a served feature shifted materially
Drift breadth	≥ 2 features flagged	drift is widespread, not noise
Rolling recall	< 0.90	too much fraud missed
Rolling precision	< 0.20	too many false alarms
PR-AUC decay	> 0.10 below reference	ranking power eroded

Table 21: Retraining triggers (`DEFAULT_TRIGGERS` in `monitoring_core.py`).

In operation, scored transactions and their eventual labels are logged, the report is regenerated on a schedule (`pixi run monitor`), and an alert fires when `retrain_recommended` is true.

11 Risk-Policy Recommendations (Module 8)

A single fraud/not-fraud threshold forces every flagged transaction into one action. In practice the platform must balance *blocking fraud* against *customer friction*, which calls for graded actions. We therefore translate the model score into a three-tier policy (Table 22), implemented in `deployment/scoring.py`.

Score band	Tier	Action	Rationale
$p < t^*$	auto-approve	clear immediately	below the cost-optimal cutoff
$t^* \leq p < b$	manual-review	analyst queue	uncertain; avoid a wrongful block
$p \geq b$	auto-block	block immediately	near-certain fraud

Table 22: Three-tier risk policy. t^* is the *deployed* model's cost-optimal threshold (≈ 0.09, read from `model.joblib` — see Section 9), applied by `scoring.py`; b is the auto-block threshold, calibrated as the smallest score at which held-out precision reaches 0.99 (≈ 0.80).

The auto-block boundary b is chosen from data rather than assumed: we take the lowest score at which precision on the held-out set reaches 0.99, so automatic blocks are near-certain while the ambiguous middle band is routed to a human rather than actioned automatically. Because a high score is driven by the balance-error footprint (`errorBalanceOrig`, Eq. 4), the policy reads naturally: a transaction whose origin balance fails to reconcile by the transferred amount is what escalates it. The tiers are surfaced by the API (`decision` $\in$ {APPROVE, REVIEW, BLOCK}) and colour-coded in the analyst queue.

12 Discussion

12.1 Why the scores are near-perfect

Random Forest Base attains PR-AUC 0.9976 with one false positive in 828,659 legitimate transactions. In real fraud detection such numbers would be implausible, and intellectual honesty requires explaining rather than celebrating them. The explanation is the data, not the model.

PaySim generates fraud through a scripted mechanism: the fraudulent agent empties the origin account. That action leaves an *almost deterministic* arithmetic footprint in the balance columns, which `errorBalanceOrig` (Eq. 4) captures directly—hence its 53.7% importance share. The decision boundary is therefore close to a rule, and any sufficiently flexible model finds it. Notably, the linear model does *not*, because the footprint is a nonlinear interaction of three columns: this is why Logistic Regression Base reaches only PR-AUC 0.5932 while both tree ensembles exceed 0.99. The gap between them is strong evidence that performance is driven by one nonlinear, near-deterministic pattern.

12.2 Assessment of label leakage

Our Base/Full design lets us bound the leakage effect rather than speculate. Two findings emerge. First, the leakage is **real**: synthetic features hold $\approx 28\%$ of XGBoost-Full importance and lift Logistic Regression by +0.38 PR-AUC. Second, it is **not load-bearing for the best models**: tree ensembles gain only +0.002 to +0.005, and the strongest honest configuration (Random Forest Base) uses no synthetic features at all. Consequently our headline conclusions do not rest on leaked information—but any Full number should be read as an optimistic upper bound.

12.3 Limitations and threats to validity

1. **Simulated data.** PaySim is agent-based, not real transactions. Its fraud is more regular, and its balance bookkeeping cleaner, than reality. Performance here should not be extrapolated to production.
2. **Synthetic features are label-derived.** Their measured strength reflects our chosen parameters (Table 2), not empirical evidence.
3. **Degenerate velocity features.** Because accounts rarely recur, the required behavioural features carry no signal, so this dimension of the risk model is untested.
4. **Random rather than temporal split.** We split randomly with stratification. A production system faces *temporal* generalization—training on the past, scoring the future—which a time-based split would evaluate more faithfully and would likely yield lower scores.
5. **Assumed friction cost.** $C_{FP} = 10$ is a modeling assumption, not a measurement.
6. **Single split, no cross-validation.** Results come from one stratified 70/30 split with a fixed seed; we report no variance across resamples.

12.4 Model recommendation

We separate the *best* model from the *deployed* model, a standard and defensible distinction:

- **Random Forest Base** is the best honest performer (PR-AUC 0.9976, one false positive, no leakage) and is what we report as the headline result.
- **XGBoost Base** is our deployment recommendation. It is statistically indistinguishable in practical terms (0.9944 PR-AUC) but yields a far more compact and faster-loading artifact, which matters for a real-time API on constrained infrastructure. It is also leakage-free.

Both decisively outperform the incumbent rule-based control (Section 4.6), which catches 16 frauds versus 2,451 for Random Forest Base.

13 Reproducibility

All results derive from a single notebook executed top-to-bottom with fixed seeds (`SEED` $= 42$ for generation, 42 for splitting and all models). The PaySim CSV is not redistributed (it exceeds GitHub's file-size limit) and must be downloaded from Kaggle; the repository README documents the procedure. The synthetic features, the data dictionary, and every table in this report are regenerated deterministically by re-running the notebook. Source code is available at https://github.com/Trungdn4-458311/FraudDetectionBE.

14 Conclusion and Future Work

We built an end-to-end fraud-detection pipeline on PaySim combined with a synthetic contextual feed, covering all eight modules: business understanding and KPI definition, exploratory analysis, documented cleaning, feature engineering with imbalance handling and feature selection, the training and evaluation of six models under a cost-based operating criterion, a real-time API with an analyst review-queue interface (Section 9), a drift and performance monitoring component with retraining triggers (Section 10), and a three-tier risk policy (Section 11).

Technically, the tree ensembles reach PR-AUC > 0.99, and the cost-optimal threshold $t^* = 0.17$ stops 99.96% of fraud value at a false-positive rate of 0.032%—a 14.7% cost reduction over the default threshold and a decisive improvement over the incumbent rule-based control.

Methodologically—and more importantly—the contribution is the **Base-versus-Full design**, which converts a latent label-leakage flaw into a measured quantity, and the accompanying finding that near-perfect scores here reflect the *easiness of the data* rather than genuine generalization. We identified the specific mechanism (a near-deterministic balance signature captured by `errorBalanceOrig`) and showed that our best honest model needs no synthetic features at all. We regard this diagnosis, rather than the headline metric, as the substantive result.

Future work. Deployment (Module 6), monitoring (Module 7) and the risk policy (Module 8) described above are now implemented; the remaining direction is *methodological hardening*: adopt a temporal split, add cross-validation with variance reporting, calibrate C_{FP} from real operational costs, and validate the pipeline on a genuine transaction dataset where the balance signature is not near-deterministic. Extending the served feature set with behavioural signals that are meaningful on real (recurring-account) data would also let the velocity/recency features finally contribute.